

UNIVERSIDAD AUTÓNOMA DE TAMAULIPAS

FACULTAD DE INGENIERÍA TAMPICO

INTEGRANTES:

**VICENTE BAUTISTA ROBERTO NICOLAS
CHAVARRÍA URIBE LEO
MORALES URRUTIA JAVIER ANTONIO
SÁNCHEZ MORALES JESÚS**

DOCENTE:

DR. DANTE ADOLFO MUÑOZ QUINTERO

ASIGNATURA:

PROGRAMACIÓN DE SISTEMAS DE BASE 1

CARRERA:

ING. EN SISTEMAS COMPUTACIONALES

GRADO Y GRUPO:

VERDAD, BELLEZA, PROBIDAD

8vo K

FECHA:

02 DE SEPTIEMBRE DEL 2026

Propósito

Home Script es un lenguaje de dominio específico (DSL) declarativo para la automatización de dispositivos y sensores del hogar (domótica). Está inspirado en la simplicidad de plataformas como Arduino, pero diseñado como un lenguaje propio, enfocado exclusivamente en describir sensores, dispositivos, parámetros de configuración y reglas de comportamiento automático (incluyendo máquinas de estado simples).

¿A quién va dirigido?

Estudiantes, hobbistas de electrónica/domótica y desarrolladores de sistemas embebidos que quieran expresar reglas de automatización de forma legible y declarativa, sin lidiar con la complejidad de un lenguaje de propósito general (gestión de memoria, sintaxis extensa, etc.).

¿Qué problema resuelve?

Separa claramente la declaración de entidades físicas (dispositivos y sensores) y sus parámetros de configuración, de la lógica condicional que define el comportamiento automático del sistema (por ejemplo, encender el clima cuando la temperatura supera un umbral), permitiendo además modelar comportamientos más complejos mediante máquinas de estado.

Ejemplo de código fuente válido

El siguiente fragmento declara dispositivos y sensores del hogar, define parámetros de umbral, aplica una regla condicional anidada, un ciclo de control y una máquina de estados simple:

```
%%writefile homescrypt_ejemplo.txt  
DISPOSITIVO clima  
DISPOSITIVO ventilador  
DISPOSITIVO luces  
SENSOR temperatura
```

SENSOR humedad

PARAMETRO frio = 18

PARAMETRO calor = 28

PARAMETRO humedad_max = 60.5

```
SI temperatura >= calor Y humedad > humedad_max {  
    ENCENDER clima  
    ENCENDER ventilador  
}  
SINO SI temperatura <= frio {  
    APAGAR clima  
    APAGAR ventilador  
}  
MIENTRAS temperatura > frio {  
    ENCENDER luces  
}  
ESTADO Noche {  
    CUANDO temperatura < frio -> Frio  
}
```

Código objeto destino y Mapeo de traducción

El compilador/traductor de Home Script tiene como objetivo de salida el código fuente en C++, compatible con el framework de Arduino, orientado a microcontroladores estándar de automatización, por ejemplo: Arduino Uno. Además, permite un mapeo limpio entre las sentencias declarativas de alto nivel y las primitivas de hardware (PinMode, digitalWrite, analogRead).

Por lo que:

- **Declaraciones (DISPOSITIVO, SENSOR):** Se traducen a la asignación de pines y su inicialización mediante pinMode() dentro de la función obligatoria de setup()
- **Configuración (PARAMETRO):** Se mapean a constantes y variables globales tipadas, por ejemplo, int, float.

- **Logica condicional (SI, SINO SI):** Se traduce a sentencias nativas: if /else if dentro del ciclo loop().
- **Ciclo (MIENTRAS):** Inicialmente, se utiliza un ciclo while de forma directa para mantener el mismo funcionamiento del código original. Posteriormente, se contempla la posibilidad de optimizarlo mediante estructuras que permitan ejecutar las operaciones sin bloquear el programa.
- **Comandos de hardware (ENCENDER, APAGAR):** Se traducen primitivas como digitalWrite(PIN, HIGH/LOW).
- **Maquinas de estado (ESTADO):** Se traducen mediante un tipo enumerado (enum) y bloques switch-case

Retomando el código anterior, el siguiente código sería la salida del código objeto producido (C++/Arduino)

```
//
=====
=====
// CÓDIGO OBJETO GENERADO AUTOMÁTICAMENTE POR EL COMPILADOR
HOMEScript
// Plataforma objetivo: C++ / Arduino Framework
=====
=====

// 1. Mapeo de pines para DISPOSITIVOS y SENSORES
const int PIN_clima = 2;
const int PIN_ventilador = 3;
const int PIN_luces = 4;
const int PIN_temperatura = A0;
const int PIN_humedad = A1;

// 2. Mapeo de PARAMETROS a variables tipadas
```

```
int frio = 18;
```

```
int calor = 28;
```

```
float humedad_max = 60.5;
```

```
// Variables de lectura para sensores
```

```
float temperatura = 0.0;
```

```
float humedad = 0.0;
```

```
// 3. Definición de la Máquina de Estados (ESTADO)
```

```
enum EstadosSistema { ESTADO_Noche, ESTADO_Frio };
```

```
EstadosSistema estadoActual = ESTADO_Noche;
```

```
void setup() {
```

```
    // Configuración de pines de salida (DISPOSITIVO)
```

```
    pinMode(PIN_clima, OUTPUT);
```

```
    pinMode(PIN_ventilador, OUTPUT);
```

```
    pinMode(PIN_luces, OUTPUT);
```

```
    // Configuración de pines de entrada (SENSOR)
```

```
    pinMode(PIN_temperatura, INPUT);
```

```
    pinMode(PIN_humedad, INPUT);
```

```
    // Estado inicial seguro para actuadores
```

```
    digitalWrite(PIN_clima, LOW);
```

```
    digitalWrite(PIN_ventilador, LOW);
```

```
    digitalWrite(PIN_luces, LOW);
```

```
}
```

```
void loop() {
```

```
    // Actualización / lectura de señales de sensores
```

```
temperatura = analogRead(PIN_temperatura);
```

```
humedad = analogRead(PIN_humedad);
```

```
// Lógica condicional: SI / SINO SI
```

```
if (temperatura >= calor && humedad > humedad_max) {
```

```
    digitalWrite(PIN_clima, HIGH);
```

```
    digitalWrite(PIN_ventilador, HIGH);
```

```
} else if (temperatura <= frio) {
```

```
    digitalWrite(PIN_clima, LOW);
```

```
    digitalWrite(PIN_ventilador, LOW);
```

```
}
```

```
// Estructura iterativa: MIENTRAS
```

```
while (temperatura > frio) {
```

```
    digitalWrite(PIN_luces, HIGH);
```

```
    // Refresco de lectura dentro del ciclo para evitar bloqueos
```

```
    temperatura = analogRead(PIN_temperatura);
```

```
}
```

```
// Evaluación de Máquina de Estados: ESTADO Noche
```

```
switch (estadoActual) {
```

```
    case ESTADO_Noche:
```

```
        // CUANDO temperatura < frio -> Frio
```

```
        if (temperatura < frio) {
```

```
            estadoActual = ESTADO_Frio;
```

```
        }
```

```
        break;
```

```
    case ESTADO_Frio:
```

```
        // Lógica correspondiente al estado derivado
```

```

break;
}

delay(50); // Muestreo periódico
}

```

Catálogo preliminar de tokens

Categoría	Patrón / Regla	Ejemplos	Notas
Palabras reservadas	Cadena exacta (mayúsculas)	DISPOSITIVO, SENSOR, PARAMETRO, SI, SINO, MIENTRAS, ENCENDER, APAGAR, ESTADO, CUANDO, Y, O, NO, VERDADERO, FALSO	Fijas; no pueden usarse como identificadores
Identificadores	[A-Za-z_][A-Za-z0-9_]*	clima, ventilador, temperatura, humedad_max, Noche	Nombres de dispositivos, sensores, parámetros y estados
Constantes enteras	\d+	18, 28, 100	Sin signo ni punto decimal
Constantes flotantes	\d+\.\d+	60.5, 21.0	Parte entera y decimal obligatorias
Constantes de cadena	"[^\"\\n]*"	"Modo noche activado"	Para mensajes/etiquetas de estado (uso futuro)
Operadores aritméticos	+ - * /	temperatura - 2	Permiten expresiones aritméticas anidadas
Operadores relacionales	> < >= <= == !=	temperatura >= calor	Comparan sensores, parámetros y literales

Operadores lógicos	Y O NO	temperatura >= calor Y humedad > humedad_max	Palabras reservadas que actúan como operadores; permiten anidar condiciones
Operador de asignación	=	PARAMETRO frio = 18	Único mecanismo de asignación/declaración con valor
Flecha de transición	->	CUANDO temperatura < frio -> Frio	Usada en bloques ESTADO para definir transiciones
Delimitadores	{ }	SI ... { ... }	Delimitan bloques de sentencias (control de flujo, estados)

Checklist de viabilidad

Criterio	¿Cumple?	Cómo lo cumple HomeScript
Al menos 3 tipos de tokens bien diferenciados	Sí	Palabras reservadas, identificadores y literales (enteros, flotantes y cadenas), más operadores y delimitadores.
Expresiones aritméticas o lógicas anidadas	Sí	Condiciones como "temperatura >= calor Y humedad > humedad_max" combinan comparaciones con operadores lógicos Y/O/NO, y admiten sub-expresiones con paréntesis.
Estructura de control de flujo o de bloques	Sí	SI/SINO condicional, MIENTRAS iterativo, y bloques ESTADO { ... } para máquinas de estado.

Mecanismo explícito de asignación o declaración	Sí	DISPOSITIVO/SENSOR declaran entidades; PARAMETRO nombre = valor asigna un valor inicial.
---	----	--

Autoevaluación Muestreada

Pregunta 1 (Conflictos Léxicos)

No existe ambigüedad entre palabras reservadas e identificadores porque todas las palabras reservadas de HomeScript se escriben completamente en mayúsculas (DISPOSITIVO, SENSOR, PARAMETRO, SI, SINO, MIENTRAS, ENCENDER, APAGAR, ESTADO, CUANDO, Y, O, NO, VERDADERO, FALSO), mientras que la convención de nombres para identificadores de usuario (dispositivos, sensores, parámetros, estados) es minúsculas o PascalCase (por ejemplo: clima, humedad_max, Noche).

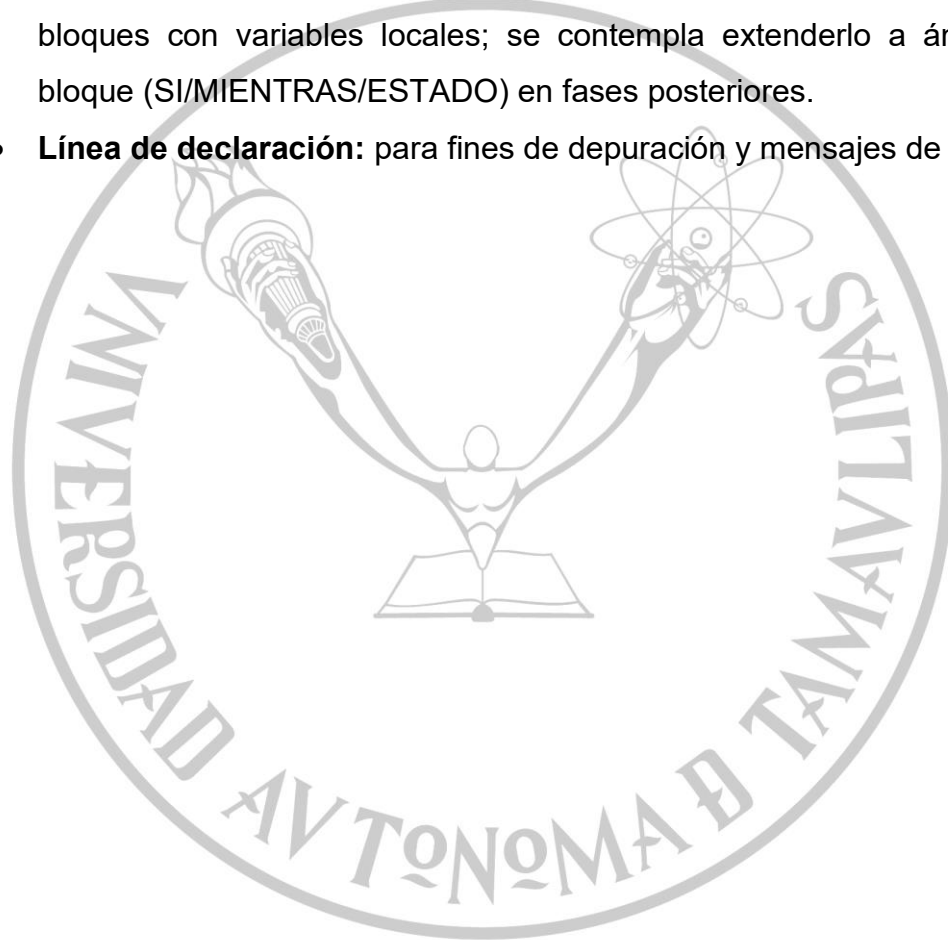
Aun así, el analizador léxico no confía únicamente en la convención de estilo: primero reconoce cualquier secuencia que cumpla el patrón de identificador `[A-Za-z_][A-Za-z0-9_]*` y, antes de emitir el token final, verifica si el lexema coincide exactamente con el conjunto cerrado de palabras reservadas. Si coincide, se reclasifica como `PALABRA_RESERVADA`; si no, se conserva como `IDENTIFICADOR`. Esto resuelve cualquier posible colisión sin necesidad de reglas adicionales, y es el enfoque ya implementado en el prototipo de tokenizador.

Pregunta 2 (Estructura de la Tabla de Símbolos)

Para los elementos de HomeScript, la Tabla de Símbolos registra los siguientes atributos por cada identificador declarado:

- **Nombre:** el identificador tal como aparece en el código (clima, temperatura, frio, etc.).
- **Categoría:** DISPOSITIVO, SENSOR o PARAMETRO, según la palabra reservada usada en su declaración.

- **Tipo de dato:** "entidad" para dispositivos/sensores (no almacenan un valor primitivo), o entero/flotante/cadena para parámetros, inferido a partir del literal asignado.
- **Valor inicial:** el valor asignado en la declaración (por ejemplo 18 para el parámetro frio); nulo para dispositivos y sensores.
- **Ámbito:** por ahora global, ya que HomeScript no define aún funciones ni bloques con variables locales; se contempla extenderlo a ámbito de bloque (SI/MIENTRAS/ESTADO) en fases posteriores.
- **Línea de declaración:** para fines de depuración y mensajes de error.



VERDAD, BELLEZA, PROBIDAD